

The Grand Palate

Complete Project Documentation

Hotel and restaurant web application for a luxury multi-cuisine venue in Bandra West, Mumbai. This document explains the architecture, every major user flow, how payments are collected and verified, and the security controls implemented in the current codebase.

Brand: The Grand Palate (Est. 2010). Stack: static HTML/CSS/JavaScript frontend, Node.js + Express API, MySQL, Razorpay Checkout, optional SMTP email, and an optional OpenAI concierge. Version described: local project hotel_palate-main as of 16 September 2026.

1. What the product does

The site is a marketing and booking surface for dining, private events, and a lounge. Guests browse Home, Menu, Reservations, Party Hall, Lounge, and Contact. They can create an account, verify email, sign in, send a contact message, chat with an on-site assistant, and pay a small reservation deposit before a table is stored as confirmed.

Administrators sign in with a user whose role is admin. The admin page only confirms that the session is an administrator; it is not a full back-office for orders or menus.

â€¢ Public brochure pages: cuisine story, opening hours, party hall, lounge, contact details.

â€¢ Account system: register, verify email, login, logout, password reset request, session cookie.

â€¢ Reservations: form data is not saved until payment succeeds (or a UPI reference is submitted for later check).

â€¢ Payments: Razorpay order creation, Checkout widget (UPI, card, netbanking, wallet), HMAC signature verification, then reservation insert.

â€¢ Contact messages stored in MySQL. AI concierge with OpenAI or a local keyword fallback.

2. Folder layout and how the app is served

The repository is split into frontend and backend. Express serves the frontend folder as static files from the same origin as the API. That matters for cookies: the gp_session cookie is first-party when the browser loads pages from the Node server (for example http://localhost:3000). Opening HTML as a file:// page or a separate static host will break login and payment verify calls unless a reverse proxy unifies the origin.

hotel_palate-main/
frontend/ HTML pages, CSS in each page, JS widgets
backend/ server.js, schema.sql, .env, package.json
SECURITY.md production hardening notes
docs/ this documentation generator and PDF

Frontend files: index.html (home + sign-in gate), menu.html, reservation.html, partyhall.html, lounge.html, contact.html, auth.html, admin.html, payment.html, verify-email.html. Scripts: auth-modal.js, auth.js, form-handler.js, payment.js, chat-widget.js.

Backend entry: backend/server.js. Start with npm start or npm run dev from backend after schema.sql is applied and .env is filled. Default port 3000. Health check: GET /api/health (SELECT 1 against MySQL).

3. Architecture

Browser pages never talk to MySQL. They call JSON APIs under /api. Express middleware order is important: Helmet headers, JSON body parser (16 KB max, strict JSON), static files, then

cookie-parser, then loadUser (attach request.user from gp_session if valid), then route handlers, then a generic 500 handler that logs api_error without leaking stack traces to the client.

MySQL is accessed with mysql2/promise connection pool (limit 10). Parameterized queries (placeholders ?) are used for user input to reduce SQL injection risk. Payment verification uses a dedicated connection, BEGIN, SELECT ... FOR UPDATE on the order row, insert reservation, update order to paid, COMMIT. Rollback on failure.

Secrets live only in backend environment variables. Frontend receives Razorpay key_id (public) after an order is created. The Razorpay key_secret never ships in HTML or JS.

4. Frontend pages in detail

4.1 Home (index.html)

Hero, cuisine cards (Indian, Italian, Chinese, Mexican), about stats, beverages, reservations / party hall / lounge teasers, footer. Scripts: chat-widget.js and auth-modal.js. The sign-in modal is a gate: if GET /api/auth/me is not OK, the dialog opens and cannot be dismissed with close, Escape, or backdrop click. The guest must submit email and password. Successful login redirects admin users to /admin.html and everyone else to /index.html. If already signed in, the modal stays hidden.

4.2 Menu, Lounge, Party Hall

Content pages with the same navigation. Chat widget is included. Sign in links go to auth.html on these pages (the modal script is currently only on the home page). Party hall describes private events; lounge describes the upper-floor beverage and music space.

4.3 Reservations (reservation.html)

Form fields: name, email, phone, date, time (select), guests (select), optional notes. form-handler.js intercepts submit. Because the form sits in .reservation-form, the client POSTs JSON to /api/payments/order, not directly to /api/reservations. On success it navigates to payment.html with query parameters order_id, amount, currency, key_id.

4.4 Payment (payment.html + payment.js)

Shows a deposit of Rs 2.00 (200 paise). Pay securely loads Razorpay Checkout. If query params are missing, amount is not 200, currency is not INR, or the Checkout script failed, the button is disabled. After the guest pays, Checkout calls handler with razorpay_order_id, razorpay_payment_id, razorpay_signature. The page POSTs those to /api/payments/verify. Success copy: Payment received and reservation confirmed. Dismissing Checkout sets Payment was cancelled and re-enables the button. A reservation is not created on cancel.

4.5 Contact

Form name, email, subject, message. form-handler.js POSTs /api/messages. Success message is shown; the form resets. No payment involved.

4.6 Auth page (auth.html)

Standalone sign-in / create-account page using auth.js. Same API as the modal. Useful for admin redirect and pages that do not load auth-modal.js. Forgot password links to /reset-password (route is not implemented as a dedicated HTML page in the frontend folder; the API for requesting and completing reset exists).

4.7 Verify email

Reads token from the URL, POSTs /api/auth/verify-email, then offers a return link.

4.8 Admin

GET `/api/admin/status` with cookies. If not admin, after a short delay redirects to `/auth.html`. Logout POSTs `/api/auth/logout` and returns to auth.

5. Authentication and accounts

Users table: email (unique, max 254), password_hash, email_verified (false until the mail link is used), role enum user or admin (default user). Admin is not self-serve; it must be set in the database.

5.1 Registration

POST `/api/auth/register`. Email lowercased. Password 12 to 128 characters. Password hashed with scrypt (N=16384, r=8, p=1, 64-byte key) plus a 16-byte random salt, stored as salt:hex. A 32-byte random verification token is created; only HMAC-SHA256(token, SESSION_SECRET) is stored. The raw token is emailed as `PUBLIC_APP_URL/verify-email?token=...`. Duplicate emails return a generic 409 so the API does not confirm which addresses exist, aside from the wording Unable to create account with those details.

5.2 Email verification

POST `/api/auth/verify-email`. Token must match a non-expired row (24 hours). User email_verified is set true and the token row is deleted. Resend: POST `/api/auth/resend-verification` always returns the same message whether or not the account exists, to limit account enumeration. Old verification tokens for that user are deleted before issuing a new one.

5.3 Login

POST `/api/auth/login`, extra limiter: 10 attempts per 15 minutes per IP. Password is verified with scrypt and timing-safe compare. Missing users still run verify against a dummy hash path conceptually by failing verifyPassword. Unverified accounts get 403 Verify your email before signing in. Success creates a session row and Set-Cookie `gp_session`.

5.4 Sessions

Raw session token: 32 random bytes hex. Stored value: HMAC-SHA256 with SESSION_SECRET (64 hex chars). Cookie flags: httpOnly (JavaScript cannot read it), sameSite=lax, path=/, maxAge 8 hours, secure=true only when NODE_ENV=production. Session row also stores IP and user-agent snapshot. loadUser joins sessions to users where expires_at is in the future. Logout deletes that token hash and clears the cookie. Password reset deletes ALL sessions for that user so stolen cookies die after a reset.

5.5 Password reset

POST `/api/auth/request-password-reset` always returns a generic success-style message. If the user exists, a 30-minute token is stored (hashed) and emailed to `PUBLIC_APP_URL/reset-password?token=...`. POST `/api/auth/reset-password` sets a new scrypt hash, deletes the reset token, and wipes sessions.

5.6 Authorization helpers

requireVerifiedUser: 401 if no session, 403 if email not verified. requireAdmin: must be verified and role admin. ownedReservation: reservation id must be digits and belong to request.user.id, else 404 (does not leak other people's bookings). GET/DELETE `/api/reservations/:id` use these guards. Creating a reservation through the unpaid POST `/api/reservations` always returns 402 Payment is required to reserve a table.

6. Payments â end-to-end

Business rule: a table is not confirmed until money is verified. The deposit amount is hardcoded as reservationFeePaise = 200, which is 200 paise = Rs 2.00 INR. That amount is a technical minimum suitable for Razorpay test mode, not a production restaurant deposit. Change the constant and the frontend checks together if you raise the fee.

6.1 Why Razorpay

Card, UPI, netbanking, and wallets are collected on Razorpay-hosted Checkout. The site never sees full card numbers or UPI PINs. The server only creates an Order and later verifies the payment signature with the merchant secret.

6.2 Step 1 â Create order

Guest submits the reservation form. Server validates name (max 120), email, phone (max 30), date YYYY-MM-DD, time HH:MM, guests integer 1-30, notes max 1000. If RAZORPAY_KEY_ID and RAZORPAY_KEY_SECRET are missing, API returns 503 Payment service is not configured.

razorpay.orders.create is called with amount 200, currency INR, a unique receipt res_<random>, and notes.email. The Razorpay order id plus guest details are inserted into payment_orders with status created, amount_paise 200, optional user_id if signed in. Response 201: orderId, amount, currency, keyId (publishable).

6.3 Step 2 â Checkout in the browser

payment.js constructs Razorpay({ key, amount, currency, order_id, name The Grand Palate, description Table reservation deposit }). Methods: upi, card, netbanking, wallet. Display config prefers a UPI block first but still shows default blocks. The guest completes payment on Razorpay. Checkout returns payment ids and a signature. No reservation row exists yet.

6.4 Step 3 â Verify signature (the important security step)

POST /api/payments/verify body: razorpay_order_id, razorpay_payment_id, razorpay_signature. The server computes HMAC-SHA256 of orderId|paymentId using RAZORPAY_KEY_SECRET and compares with crypto.timingSafeEqual. If a client faked a payment id without paying, the HMAC will not match and the API returns Payment verification failed. Never trust Checkout success alone without this check.

If the HMAC is valid, a transaction locks the payment_orders row FOR UPDATE. The row must exist, status must still be created, and amount_paise must still equal 200. That blocks replay: a second verify cannot insert a second reservation for the same paid order. Then INSERT reservations copying guest fields plus payment_order_id and payment_id, UPDATE payment_orders to status paid, payment_id, paid_at=NOW(), COMMIT. Response 201 with reservation id and confirmation text.

6.5 Alternate path â UPI reference without Checkout verify

POST /api/payments/upi-confirmation accepts order_id and utr (6-50 alphanumeric or hyphen). If the order is still created and amount matches, status becomes upi_pending and utr is stored. HTTP 202: reservation will be confirmed after payment is checked. This path does NOT insert into reservations and does not talk to Razorpay. It is a manual-review holding state. The current payment.html UI does not call this endpoint; Checkout verify is the live guest path. Staff would need a separate process to mark upi_pending orders paid.

6.6 Payment_orders status machine

â created â Razorpay order exists, guest has not completed a verified payment.

â upi_pending â guest submitted a UTR; waiting for human or ops confirmation (no auto-reservation).

â paid â HMAC verified; reservation row created; paid_at set.

â€¢ failed â€¢ enum exists for future use; verify/UPI handlers do not currently set failed on cancel.

6.7 What is stored vs not stored

Stored: Razorpay order_id, payment_id after success, amount in paise, guest contact and booking slot, optional UTR. Not stored: card PAN, CVV, UPI PIN, full Checkout payloads. PCI scope stays with Razorpay. The publishable key_id in the query string is expected for Checkout; the secret stays on the server.

6.8 Frontend amount check

payment.js requires amount === 200. A tampered query string cannot lower the charged amount because Razorpay charges the server-created order, and verify re-reads amount_paise from the database. Tampering the query only breaks the button or shows a mismatch; it cannot confirm a reservation without a matching paid order.

7. Reservations and contact data

reservations columns: name, email, phone, reservation_date, reservation_time, guests, notes, optional user_id (SET NULL if user deleted), payment_order_id unique, payment_id, created_at. A signed-in guest has user_id copied from the payment order at verify time. Anonymous guests can still pay; user_id is null.

GET /api/reservations/:id returns the row only to the owning verified user. DELETE removes only that user's row. There is no public list of all bookings. messages table stores contact form submissions (name, email, subject, message, optional user_id unused in the current insert).

8. AI concierge

chat-widget.js is a floating Ask AI panel. History is kept in the browser (last messages sent with the request; server keeps only the last 6). POST /api/assistant, limiter 20 requests / 15 minutes. Message required; history items must be role user or assistant and content length <= 500. If OPENAI_API_KEY is empty, a keyword fallback answers reserve/book, hours/open, party/event, menu/food, else a generic line. Mode in JSON is fallback or ai.

With a key, the server calls OPENAI_API_URL (default OpenAI chat completions) with model OPENAI_MODEL (default gpt-4o-mini), temperature 0.4, max_tokens 250. System prompt: concierge for The Grand Palate; only reservations, menu, hours, lounge, party hall, contact; never invent prices or availability. Failures return 500 with a generic error. The widget displays assistant text with textContent (not innerHTML), which avoids HTML injection from model output in the chat bubble.

9. Security methods in depth

9.1 Transport and headers

X-Powered-By is disabled. Helmet is enabled (CSP is currently off so inline scripts and Razorpay CDN can run; HSTS is on only in production). Production should sit behind TLS-terminating reverse proxy. TRUST_PROXY=true only when the proxy sets client IP correctly, otherwise rate limits and session IP logs can be spoofed or pinned to the proxy.

9.2 Secrets and configuration

SESSION_SECRET must be at least 32 characters or the process refuses to start. Used as HMAC key for session, email verify, and password-reset tokens so a stolen database of hashes is not enough to reconstruct the raw tokens without the secret. Database password, SMTP password, OpenAI key, Razorpay secret belong in environment or a secret manager, never in frontend files or git. SECURITY.md tells

operators not to run MySQL as a public root account.

9.3 Password storage

scrypt is a memory-hard KDF. Salt is per-user. Timing-safe comparison avoids leaking how many hash bytes matched. Length policy 12-128 reduces trivial passwords; there is no complexity regex beyond length. Plain passwords are not logged.

9.4 Token hashing

Email verify, password reset, and session identifiers are stored as HMAC-SHA256 hex, not as the token sent to the user. A database dump of token_hash values cannot be used as cookies or links without SESSION_SECRET (and even then HMAC is not designed to reverse). Tokens are high-entropy `crypto.randomBytes(32)`.

9.5 Cookies

httpOnly blocks XSS from reading the session cookie via `document.cookie`. `sameSite=lax` reduces CSRF on cross-site POSTs from other sites (top-level GET navigations still send the cookie). secure in production requires HTTPS. 8-hour absolute expiry. Logout and password reset invalidate server-side rows, not only the browser cookie.

9.6 CSRF and same origin

There is no separate CSRF token. Protection relies on SameSite=lax cookies plus JSON APIs that are not simple form posts from other origins (CORS is not opened to arbitrary sites). If the frontend is hosted on another domain, cookies will not attach and you would need an explicit CORS and CSRF design. Keep API and pages on one origin as the code assumes.

9.7 Injection and validation

JSON body limit 16kb. Strict JSON parsing. `requiredText` rejects non-strings, empty trim, and over-max length. Email regex is basic. Dates and times are regex-checked. Reservation guests capped at 30. SQL uses bound parameters. Assistant history is filtered by role and length. Chat UI uses `textContent`. Generic error messages on auth failures and many 400s reduce oracle leaks; login still distinguishes unverified email after a correct-looking login path.

9.8 Rate limiting

â€¢ All /api: 120 requests / 15 minutes (express-rate-limit draft-8 headers).

â€¢ Login: 10 / 15 minutes (credential stuffing resistance).

â€¢ Register, verify, resend, password reset: 10 / 60 minutes.

â€¢ Assistant: 20 / 15 minutes (cost and abuse).

â€¢ Payment order/verify/UPI also sit behind the global /api limiter.

9.9 Payment security

Server-side amount, HMAC verification, row lock, single-use created status, no card data at rest. Do not confirm reservations from client-only success events. Rotate RAZORPAY_KEY_SECRET if leaked; old signatures would still verify until Razorpay rotates the key pair. Use `rzp_test_` keys in development and live keys only in production with HTTPS.

9.10 Privacy and logging

logSecurity writes JSON lines: timestamp, event, ip, method, path, userId, plus extra fields such as email on login. Useful for alerts; treat logs as sensitive. Do not log passwords, Razorpay signatures, or full payment objects. Emails contain raw tokens; HTTPS on PUBLIC_APP_URL and SMTP auth matter. In development without SMTP, verification emails are skipped with a console warning;

production throws if SMTP is missing so accounts cannot be created without a verification channel.

9.11 Account enumeration

Resend verification and password-reset request use the same response whether the email exists. Registration duplicate is slightly less generic (409). Login uses Invalid email or password for bad credentials.

9.12 Known gaps (honest)

Content-Security-Policy is disabled, so XSS in any inline or third-party script has a wider blast radius. There is no CSRF token for same-site JSON posts. Admin role is a DB flag with no audit UI. UPI pending is not auto-reconciled. reset-password HTML page is not in the frontend list. Home sign-in gate does not apply to every HTML page. Cookie-parser is mounted after static middleware (API still works). No captcha on register or contact. OpenAI key if present is a server-side secret but prompt injection can still make the model ignore the system prompt; the fallback and the instruction not to invent prices are mitigations, not guarantees.

10. Database schema summary

users: id, email unique, password_hash, email_verified, role user|admin, created_at.

sessions: token_hash PK, user_id FK cascade delete, expires_at, ip_address, user_agent, created_at.

email_verification_tokens / password_reset_tokens: token_hash PK, user_id, expires_at.

payment_orders: order_id PK (Razorpay id), guest fields, amount_paise, status, payment_id, utr, paid_at, user_id optional.

reservations: booking fields plus payment_order_id unique and payment_id after pay.

messages: contact inbox. Apply backend/schema.sql on a dedicated app account. Existing databases can use backend/migration-add-payments.sql to add payment columns and the payment_orders table.

11. API catalogue

â€¢ GET /api/health â€¢ DB ping.

â€¢ POST /api/auth/register â€¢ create user, send verify mail.

â€¢ POST /api/auth/login â€¢ session cookie.

â€¢ POST /api/auth/verify-email â€¢ token in body.

â€¢ POST /api/auth/resend-verification â€¢ email body.

â€¢ POST /api/auth/request-password-reset â€¢ email body.

â€¢ POST /api/auth/reset-password â€¢ token + new password.

â€¢ POST /api/auth/logout â€¢ 204.

â€¢ GET /api/auth/me â€¢ current verified user.

â€¢ GET /api/admin/status â€¢ admin only.

â€¢ POST /api/assistant â€¢ chat.

â€¢ POST /api/payments/order â€¢ start Razorpay order from reservation fields.

â€¢ POST /api/payments/verify â€¢ HMAC, create reservation.

â€¢ POST /api/payments/upi-confirmation â€¢ store UTR, upi_pending.

â€¢ POST /api/reservations â€¢ always 402 if body valid.

â€¢ GET/DELETE /api/reservations/:id â€¢ owner only.

â€¢ POST /api/messages â€¢ contact form.

â€¢ GET /verify-email â€¢ serves verify-email.html.

12. Environment variables

Copy backend/.env.example to backend/.env. PORT, NODE_ENV, SESSION_SECRET (32+ random chars), PUBLIC_APP_URL (real HTTPS origin in production), TRUST_PROXY, DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, OPENAI_API_KEY optional, OPENAI_MODEL, OPENAI_API_URL, SMTP_HOST, SMTP_SECURE, SMTP_USER, SMTP_PASSWORD, EMAIL_FROM, RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET are all filled in .env. Rotate all secrets if a copy leaks.

13. How to run locally

â€¢ Install MySQL. Create database grand_palate. Run schema.sql as a least-privilege user.

â€¢ cd backend && npm install. Fill .env including SESSION_SECRET and Razorpay test keys if you need payments.

â€¢ npm start. Open http://localhost:3000 (not a random static port, or cookies and /api paths will fail).

â€¢ Register, complete email verify (or set email_verified=1 in MySQL in a private lab only), sign in, submit a reservation, pay in Razorpay test Checkout, confirm the reservations row.

â€¢ To make an admin: UPDATE users SET role='admin' WHERE email='you@example.com'; then sign in and open /admin.html.

14. Production checklist

Follow SECURITY.md: private MySQL, TLS at the proxy, NODE_ENV=production, unique SESSION_SECRET, SMTP working, PUBLIC_APP_URL exact origin, firewall so only the app reaches MySQL, live Razorpay keys, raise reservationFeePaise if Rs 2 is only for tests, turn on HSTS via Helmet in production, consider enabling a CSP that still allows checkout.razorpay.com, structured log shipping, backup MySQL, and rotate secrets after incidents.

15. Guest journeys (quick reference)

Browse home: modal asks for login unless session exists. Create account: register, email link, then login. Book a table: fill reservation, server creates Razorpay order and payment_orders row, Checkout, verify HMAC, reservations row status paid. Cancel Checkout: no reservation. Contact: message stored immediately. Ask AI: widget to /api/assistant. Admin: role flag plus /api/admin/status.

16. Document scope

This PDF describes the application as implemented in this repository. It is not Razorpay legal documentation, PCI-DSS certification, or a penetration-test report. Payment amounts, SMTP, and AI behaviour depend on environment configuration. Keep secrets out of copies of this file if you paste live key names from a private .env.